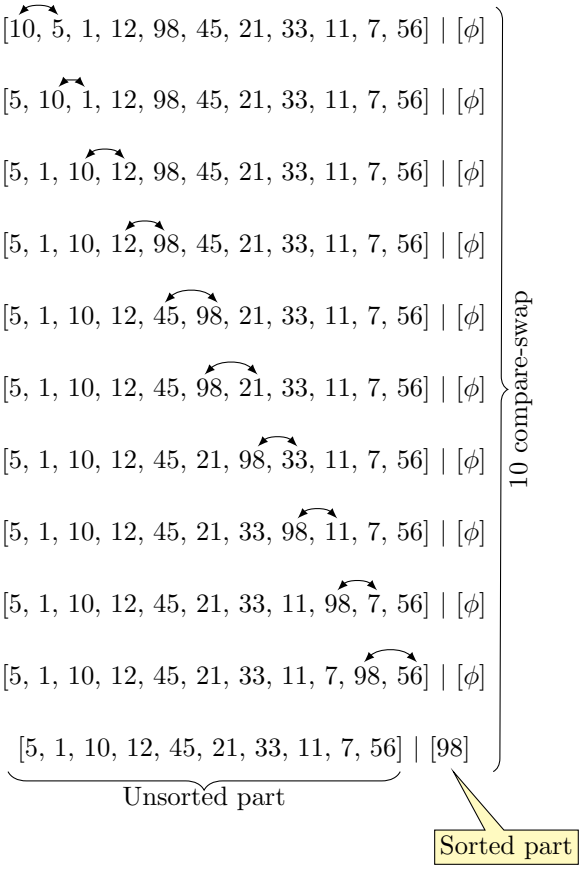
Bubble sort

Bubble sort operates on the principle that the heaviest element sinks to the bottom of a water tank, while the lightest element bubble up to the top. In this context, we consider the bottommost element or the rightmost element of the unsorted array as already sorted. By comparing this element with its adjacent one, we can swap them if necessary, allowing the heaviest element to sink further down (to the rightmost position).

The figure below illustrates the bubble sorting technique applied to a given array.



The figure shows that we establish a logical partition in the array, using a vertical bar. The bar separates the unsorted portion on the left from the sorted portion on the right. The initial configuration is illustrated in the top row. Sorted part of the array is empty. An iteration of the bubble sort starts with the leftmost

element. We compare and swap adjacent elements, specifically swapping the pair if the first element is larger than the second. This process is repeated for each subsequent pair. More specifically, after completing the swap between elements at positions i and $i + 1$, we proceed to swap the elements at positions $i + 1$ and $i + 2$, continuing this method for each pair where $1 \leq i \leq n - 1$.

The proof of correctness relies on the fact that after performing $n-1$ compare-and-swap operations, the index of the largest remaining element is $n - 1$. The maximum element can appear anywhere in the array. Each iteration starts from the beginning of the array, specifically from index one, and continues to perform compare-and-swap operations for each adjacent pair of elements. Let the current element be denoted by x. A compare-and-swap pushes the larger of the two elements to the right. Therefore, an iteration of bubble sort effectively pushes all elements smaller x to its left until it encounters an element larger than x. The larger element then becomes the element that moves to the right pushing all smaller elements to its left. The iteration concludes after completing the compare-and-swap with the last pair, which consists of the elements at index positions $n - 1$ and n. Therefore, it indicates that the largest of the remaining elements has been moved to the end of the array.

In the worst-case scenario for iteration i, $n - i$ comparisons and swaps may be necessary. Therefore, if we sum this over all iterations, the total number of comparison and swap operations is $O(n^2)$.